System Architecture

Document Information

Project: Blogging System API

Version: 1.0

Introduction

This document describes the software architecture of the Blogging System API.

The application follows the principles of **Clean Architecture**, ensuring a clear separation of responsibilities between business logic, infrastructure, persistence, and presentation layers.

The primary architectural goals are:

- Maintainability
- Scalability
- Testability
- Security
- Separation of Concerns
- Extensibility

Architectural Style

The Blogging System API adopts a **Layered Clean Architecture**.

The architecture separates the application into independent projects, each with a single responsibility.

```
                Client
                   │
                   ▼
              Blog.API
                   │
                   ▼
         Blog.Application
                   │
                   ▼
```

```
            Blog.Domain
                  ▲
                  │
        Blog.Infrastructure
```

Dependency Rule

Dependencies always point inward.

- The **Domain** layer has no dependencies on other application layers.
- The **Application** layer depends only on the **Domain**.
- The **Infrastructure** layer depends on **Application** and **Domain**.
- The **API** layer depends on **Application** and **Infrastructure**.

No outer layer may introduce business rules into an inner layer.

Project Structure

```
BlogSystem/
│
├── src/
│   ├── Blog.API/
│   ├── Blog.Application/
│   ├── Blog.Domain/
│   └── Blog.Infrastructure/
│
├── tests/
│   ├── Blog.UnitTests/
│   └── Blog.IntegrationTests/
│
├── docs/
│
├── postman/
│
├── .gitignore
│
├── README.md
│
└── BlogSystem.sln
```

Layer Responsibilities

1. Blog.API

The API layer is responsible for exposing HTTP endpoints.

Responsibilities include:

- Controllers
- Route definitions
- Authentication middleware
- Authorization middleware
- Swagger configuration
- Dependency Injection configuration
- Global exception middleware
- Request and response handling

The API layer shall not contain business logic.

---

2. Blog.Application

The Application layer contains all application use cases.

Responsibilities include:

- Business rules
- Commands
- Queries
- DTOs
- Interfaces
- Validation
- Authentication services
- Email services
- Token generation
- Mapping
- Use case orchestration

The Application layer coordinates operations without knowing implementation details.

---

3. Blog.Domain

The Domain layer represents the core business model.

Responsibilities include:

- Entities
- Enums
- Value Objects (if required)
- Domain constants
- Domain exceptions
- Business invariants

The Domain layer contains no database or framework-specific code.

---

4. Blog.Infrastructure

The Infrastructure layer provides implementations for services defined in the Application layer.

Responsibilities include:

- Entity Framework Core
- PostgreSQL
- DbContext
- Repositories
- SMTP Email Service
- JWT implementation
- BCrypt password hashing
- Refresh token persistence
- OpenTelemetry configuration
- External integrations

Request Lifecycle

Every HTTP request shall follow the same processing pipeline.

```
Client
   │
   ▼
Controller
   │
   ▼
Application Service
   │
   ▼
Repository
   │
```

```
   ▼
Entity Framework Core
   │
   ▼
PostgreSQL
   │
   ▲
Repository
   │
Application Service
   │
Controller
   │
Client
```

This ensures that controllers remain lightweight and business logic is centralized.

Authentication Flow

The authentication process follows this sequence:

Registration

1. User submits registration details.
2. Password is validated.
3. Password is hashed using BCrypt.
4. User is stored as **Unverified**.
5. Six-digit OTP is generated.
6. OTP is stored securely.
7. OTP is sent via SMTP.
8. User verifies the OTP.
9. Account status becomes **Verified**.

Login

1. User submits email and password.
2. Password hash is verified.
3. Verification status is checked.
4. JWT Access Token is generated.
5. Refresh Token is generated.
6. Tokens are returned to the client.

Forgot Password

1. User requests password reset.
2. OTP is generated.
3. OTP is emailed.
4. User submits OTP.
5. OTP is validated.
6. User provides a new password.
7. Password is hashed and updated.

Authorization

The system uses **Role-Based Access Control (RBAC)**.

Supported roles:

- Administrator
- User

Administrator Permissions

- Create posts
- Edit own posts
- Delete own posts
- Save drafts
- Publish drafts
- Lock and unlock comments
- Reply to comments on owned posts
- Delete any comment for moderation

User Permissions

- Register
- Verify account
- Login
- Read published posts
- Comment
- Reply to comments
- Delete own comments
- Change password
- Reset forgotten password

Database Access

The application uses **Entity Framework Core** as its Object-Relational Mapper (ORM).

Database operations are performed through the Infrastructure layer.

Business logic shall never interact directly with the database.

Data Storage

Persistent data shall be stored in PostgreSQL.

The database stores:

- Users
- Blog Posts
- Comments
- Refresh Tokens
- Email Verification OTPs
- Password Reset OTPs

Logging and Observability

The application uses **OpenTelemetry** to collect telemetry data.

Instrumentation includes:

- HTTP requests
- Database operations
- Exceptions
- Custom application traces

Telemetry is exported to the console during development.

Security Architecture

Security measures include:

- BCrypt password hashing
- JWT authentication

- Refresh Tokens
- Role-Based Authorization
- Email OTP verification
- Password reset OTP
- Input validation
- Secure configuration management

Sensitive information shall never be logged or exposed in API responses.

Error Handling

A global exception handling middleware shall intercept unhandled exceptions.

The middleware shall:

- Log the exception
- Return a standardized JSON response
- Prevent internal implementation details from being exposed

Validation

Incoming requests shall be validated before business logic executes.

Validation includes:

- Required fields
- Email format
- Password complexity
- String length
- Business rule validation

Invalid requests shall return appropriate HTTP 400 responses with descriptive validation messages.

API Design Principles

The API follows RESTful principles.

Characteristics include:

- Resource-oriented endpoints
- Stateless communication

- JSON request and response bodies
- Appropriate HTTP status codes
- Consistent response structure

Example endpoints:

```
/api/auth/register
/api/auth/login
/api/auth/verify-email
/api/auth/forgot-password
/api/auth/reset-password

/api/posts
/api/posts/{id}

/api/comments
/api/comments/{id}
```

Dependency Injection

Dependency Injection shall be used throughout the application.

All services shall be registered during application startup.

This promotes loose coupling and simplifies testing.

Testing Strategy

Testing shall occur incrementally.

Each completed feature shall undergo:

- Successful project build
- Database verification
- API testing with Postman
- Manual functional verification

No new feature shall begin until the current feature has passed testing.

Future Extensibility

The architecture is designed to support future enhancements, including:

- Image uploads
- Categories
- Tags
- Full-text search
- Cloud deployment
- Containerization
- Caching
- Rate limiting
- API versioning
- Third-party authentication providers

These enhancements can be introduced without significant architectural changes.

Architectural Principles Summary

The Blogging System API shall be:

- Modular
- Secure
- Testable
- Maintainable
- Extensible
- Observable
- Well-documented

Every implementation decision shall align with these principles to ensure long-term maintainability and code quality.